Northeastern University

EECE 7205
Fundamentals of Computer Engineering

Stacks and Queues — In-Class Work

Professor: Dr. Fatema Nafa

Rohini Bidnurmath
NUID: 003916201
Catalina Buitano
NUID: 002330106

October 2026

Contents

Q1: Expression Evaluation Using Stacks **2**
(a) Implementation . . . 2
(b) Evaluation . . . 3
(c) Trace . . . 3
(d) Associativity and Division Tests . . . 4

Q2: Logic Gate Analysis and Implementation **5**
(a) Circuit . . . 5
(b) Boolean Expression . . . 5
Implementing Z with a Stack . . . 5
(c) Truth Table . . . 8
(d) Karnaugh Map and Minimum SOP . . . 8
(e) NAND-Only Implementation . . . 9
(f) Worst-Case Propagation Delay . . . 10

Q1: Expression Evaluation Using Stacks

(a) Implementation

Two stacks are used: *values* (integers) and *ops* (operators). The expression is a vector of string tokens.

- **Number** (an optional leading `-` followed by digits, e.g. `-7`): push onto *values*. A lone `-` is the subtraction operator.
- **Operator** *op*: while the top of *ops* has *higher* precedence than *op*, or *equal* precedence and *op* is left-associative, pop one operator and two values, apply it, and push the result. Then push *op*.
- After the last token, apply all remaining operators.

Precedence: `^` (3, right-associative) > `*`, `/` (2) > `+`, `-` (1). Because `^` is right-associative, an incoming `^` does *not* pop an equal-precedence `^`. Integer division in C++ truncates toward zero, so FloorDiv subtracts 1 from the quotient when the remainder is nonzero and the operands have opposite signs.

Algorithm 1 Helper functions

```
1:  function IsNumber(tok)
2:      i ← 0
3:      if tok[0] = '-' then
4:          if |tok| = 1 then return false                ▷ lone minus is an operator
5:          i ← 1
6:      return true if every character tok[i..] is a digit, else false

7:  function Precedence(op)
8:      if op ∈ {+, −} then return 1
9:      if op ∈ {∗, /} then return 2
10:     if op = ^ then return 3

11: function FloorDiv(a, b)
12:     q ← a/b                                           ▷ truncates toward 0
13:     if a mod b ≠ 0 and (a < 0) ≠ (b < 0) then
14:         q ← q − 1
15:     return q

16: function ApplyOp(a, b, op)
17:     if op = + then return a + b
18:     else if op = − then return a − b
19:     else if op = ∗ then return a · b
20:     else if op = / then return FloorDiv(a, b)
21:     else if op = ^ then
22:         r ← 1; repeat b times: r ← r · a; return r
```

Algorithm 2 Evaluate an infix expression with two stacks

```
1: procedure ApplyTop(values, ops)
2:     op ← ops.pop();  b ← values.pop();  a ← values.pop()
3:     values.push(ApplyOp(a, b, op))

4: function Evaluate(tokens)
5:     values ← empty stack;  ops ← empty stack
6:     for each tok in tokens do
7:         if IsNumber(tok) then
8:             values.push(toInt(tok))
9:         else
10:            while ops not empty and (Precedence(ops.top) > Precedence(tok) or
    (Precedence(ops.top) = Precedence(tok) and tok ≠ ^)) do
11:                ApplyTop(values, ops)
12:            ops.push(tok)
13:    while ops not empty do
14:        ApplyTop(values, ops)
15:    return values.top
```

The full runnable C++ version is in `q1_stack_eval.cpp`.

(b) Evaluation

$$8 + 3 \times 2^2 - 20/4 \;=\; 8 + 3 \times 4 - 5 \;=\; 8 + 12 - 5 \;=\; \boxed{15}$$

(c) Trace

Stacks are written bottom → top.

Step	Token	Action	`values`	`ops`
1	`8`	push value	[8]	[ ]
2	`+`	push op	[8]	[+]
3	`3`	push value	[8, 3]	[+]
4	`*`	prec(+) < prec(*): push	[8, 3]	[+, *]
5	`2`	push value	[8, 3, 2]	[+, *]
6	`^`	prec(*) < prec(^): push	[8, 3, 2]	[+, *, ^]
7	`2`	push value	[8, 3, 2, 2]	[+, *, ^]
8	`-`	$2^2 = 4$; $3 \cdot 4 = 12$; $8 + 12 = 20$; push −	[20]	[−]
9	`20`	push value	[20, 20]	[−]
10	`/`	prec(−) < prec(/): push	[20, 20]	[−, /]
11	`4`	push value	[20, 20, 4]	[−, /]

At step 8, the incoming `-` first pops `^` and `*` (higher precedence) and then `+` (equal precedence, and `-` is left-associative).

Remaining calculations (tokens exhausted):

Operation	`values`	`ops`
$20/4 = 5$	[20, 5]	[−]
$20 - 5 = 15$	[15]	[ ]

Final result: $\boxed{15}$.

(d) Associativity and Division Tests

- `2 ^ 3 ^ 2`: when the second `^` arrives, the top of `ops` is `^` with equal precedence, but `^` is right-associative, so nothing is popped. Stacks become values $= [2, 3, 2]$, ops $= [\hat{}, \hat{}]$. Unwinding: $3^2 = 9$, then $2^9 = \boxed{512}$ (not $(2^3)^2 = 64$).
- `-7 / 2`: `-7` is read as a single negative integer token. $-7/2 = -3.5$, and rounding toward $-\infty$ gives $\boxed{-4}$. (Plain C++ `/` would give -3; FloorDiv corrects it, since the remainder is nonzero and the signs differ.)

Output of `q1_stack_eval.cpp`:

```
2 ^ 3 ^ 2 = 512   (expected 512) PASS
-7 / 2 = -4   (expected -4) PASS
7 / -2 = -4   (expected -4) PASS
7 / 2 = 3   (expected 3) PASS
10 - 3 - 2 = 5   (expected 5) PASS
8 + 3 * 2 ^ 2 - 20 / 4 = 15   (expected 15) PASS
```

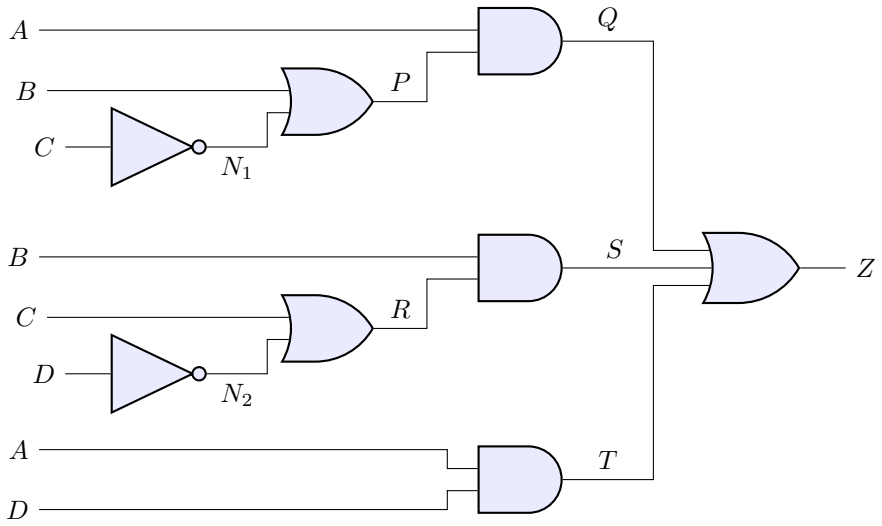
Q2: Logic Gate Analysis and Implementation

Approach. The table and Figure 1 are read gate by gate to build a Boolean expression for Z. That expression is then implemented the same way as Q1: it is written as a list of tokens, converted from infix to postfix with an *operator stack*, and evaluated with a *value stack* for every combination of A, B, C, D to produce the truth table. The K-map is then used to simplify it.

Boolean operators and their stack precedence:

Operator	Token	Precedence	Type
NOT ($\overline{X}$)	`!`	3 (highest)	unary, prefix
AND ($X \cdot Y$)	`&`	2	binary, left-assoc.
OR ($X + Y$)	`|`	1 (lowest)	binary, left-assoc.

(a) Circuit



(b) Boolean Expression

Working forward through the table:

$$N_1 = \overline{C}, \qquad N_2 = \overline{D}$$
$$P = B + \overline{C}, \qquad Q = A\,P = A(B + \overline{C})$$
$$R = C + \overline{D}, \qquad S = B\,R = B(C + \overline{D})$$
$$T = AD$$

$$Z = Q + S + T = \boxed{A(B + \overline{C}) + B(C + \overline{D}) + AD} = AB + A\overline{C} + BC + B\overline{D} + AD$$

Implementing Z with a Stack

Step 1 — Tokenize.

Writing each gate output in parentheses gives the infix token list

```
( A & ( B | !  C ) ) | ( B & ( C | !  D ) ) | ( A & D )
```

Step 2 — Infix to postfix (operator stack).

Algorithm 3 Convert a Boolean infix expression to postfix

```
1:  function ToPostfix(infix)
2:      output ← empty list;  ops ← empty stack
3:      for each tok in infix do
4:          if tok is a variable (A–D) then
5:              append tok to output
6:          else if tok = '(' or tok = '!' then
7:              ops.push(tok)                          ▷ NOT is unary prefix: never pops
8:          else if tok = ')' then
9:              while ops.top ≠ '(' do
10:                 append ops.pop() to output
11:             ops.pop()                                              ▷ discard '('
12:         else                                                       ▷ binary & or |
13:             while ops not empty and ops.top ≠ '(' and Precedence(ops.top) ≥ Prece-
    dence(tok) do
14:                 append ops.pop() to output
15:             ops.push(tok)
16:     while ops not empty do
17:         append ops.pop() to output
18:     return output
```

Trace of ToPostfix on the expression for Z (stacks bottom → top):

Token	Output (postfix so far)	Operator stack
(		[(]
A	A	[(]
&	A	[(&]
(	A	[(& (]
B	A B	[(& (]
|	A B	[(& (|]
!	A B	[(& (| !]
C	A B C	[(& (| !]
)	A B C ! |	[(&]
)	A B C ! | &	[]
|	A B C ! | &	[|]
(	A B C ! | &	[| (]
B	A B C ! | & B	[| (]
&	A B C ! | & B	[| (&]
(	A B C ! | & B	[| (& (]
C	A B C ! | & B C	[| (& (]
|	A B C ! | & B C	[| (& (|]
!	A B C ! | & B C	[| (& (| !]
D	A B C ! | & B C D	[| (& (| !]
)	A B C ! | & B C D ! |	[| (&]
)	A B C ! | & B C D ! | &	[|]
|	A B C ! | & B C D ! | & |	[|]

continued on next page

Token	Output (postfix so far)	Operator stack
`(`	`A B C ! | & B C D ! | & |`	`[| (]`
`A`	`A B C ! | & B C D ! | & | A`	`[| (]`
`&`	`A B C ! | & B C D ! | & | A`	`[| ( &]`
`D`	`A B C ! | & B C D ! | & | A D`	`[| ( &]`
`)`	`A B C ! | & B C D ! | & | A D &`	`[|]`
`end`	`A B C ! | & B C D ! | & | A D & |`	`[]`

Postfix: `A B C ! | & B C D ! | & | A D & |`

Notice the postfix groups exactly match the gates: `A B C ! | &` is Q, `B C D ! | &` is S, and `A D &` is T.

Step 3 — Evaluate postfix (value stack).

Algorithm 4 Evaluate a postfix Boolean expression

```
1:  function EvalPostfix(postfix, A, B, C, D)
2:      values ← empty stack
3:      for each tok in postfix do
4:          if tok is a variable then
5:              values.push(value of tok)
6:          else if tok = '!' then
7:              x ← values.pop();  values.push(not x)
8:          else
9:              y ← values.pop();  x ← values.pop()
10:             values.push(x and y) if tok = & else values.push(x or y)
11:     return values.top

12: procedure TruthTable(postfix)
13:     for m ← 0 to 15 do
14:         A ← bit 3 of m;  B ← bit 2;  C ← bit 1;  D ← bit 0
15:         print m, A, B, C, D, EvalPostfix(postfix, A, B, C, D)
```

Example trace for $A = 1,\ B = 0,\ C = 1,\ D = 0$ (row $m = 10$):

Token	Value stack	Token	Value stack
`A`	`[1]`	`!`	`[0 0 1 1]`
`B`	`[1 0]`	`|`	`[0 0 1]`
`C`	`[1 0 1]`	`&`	`[0 0]`
`!`	`[1 0 0]`	`|`	`[0]`
`|`	`[1 0]`	`A`	`[0 1]`
`&`	`[0]`	`D`	`[0 1 0]`
`B`	`[0 0]`	`&`	`[0 0]`
`C`	`[0 0 1]`	`|`	`[0]`
`D`	`[0 0 1 0]`		

The single value left on the stack is $Z = 0$, which matches the gates: $Q = 1 \cdot (0 + 0) = 0$, $S = 0$, $T = 1 \cdot 0 = 0$.

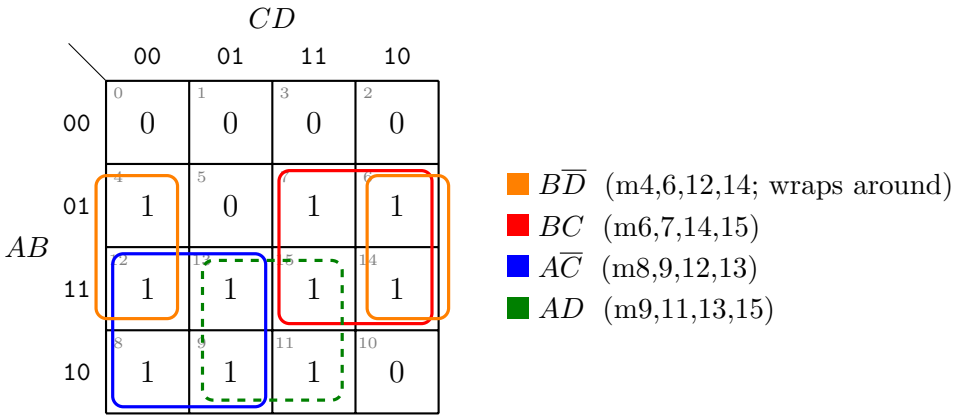
(c) Truth Table

Each row is produced by setting A, B, C, D and running EvalPostfix on the postfix form of Z (the intermediate gate signals are listed too so the table can be checked against the figure).

m	A	B	C	D	N_1	N_2	P	Q	R	S	T	Z
0	0	0	0	0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	0	0	0	0	0
2	0	0	1	0	0	1	0	0	1	0	0	0
3	0	0	1	1	0	0	0	0	1	0	0	0
4	0	1	0	0	1	1	1	0	1	1	0	**1**
5	0	1	0	1	1	0	1	0	0	0	0	0
6	0	1	1	0	0	1	1	0	1	1	0	**1**
7	0	1	1	1	0	0	1	0	1	1	0	**1**
8	1	0	0	0	1	1	1	1	1	0	0	**1**
9	1	0	0	1	1	0	1	1	0	0	1	**1**
10	1	0	1	0	0	1	0	0	1	0	0	0
11	1	0	1	1	0	0	0	0	1	0	1	**1**
12	1	1	0	0	1	1	1	1	1	1	0	**1**
13	1	1	0	1	1	0	1	1	0	0	1	**1**
14	1	1	1	0	0	1	1	1	1	1	0	**1**
15	1	1	1	1	0	0	1	1	1	1	1	**1**

$$Z(A, B, C, D) = \sum m(4, 6, 7, 8, 9, 11, 12, 13, 14, 15)$$

(d) Karnaugh Map and Minimum SOP



Prime implicants: $B\overline{D}$, BC, $A\overline{C}$, AD, and AB.

- m_4 is covered only by $B\overline{D} \Rightarrow B\overline{D}$ is essential.
- m_7 is covered only by $BC \Rightarrow BC$ is essential.
- m_8 is covered only by $A\overline{C} \Rightarrow A\overline{C}$ is essential.
- m_{11} is covered only by $AD \Rightarrow AD$ is essential.

These four essential prime implicants cover all ten minterms, so AB (m12–15) is redundant (it is the consensus term of $A\overline{C}$ and BC).

$$\boxed{Z = B\overline{D} + BC + A\overline{C} + AD}$$

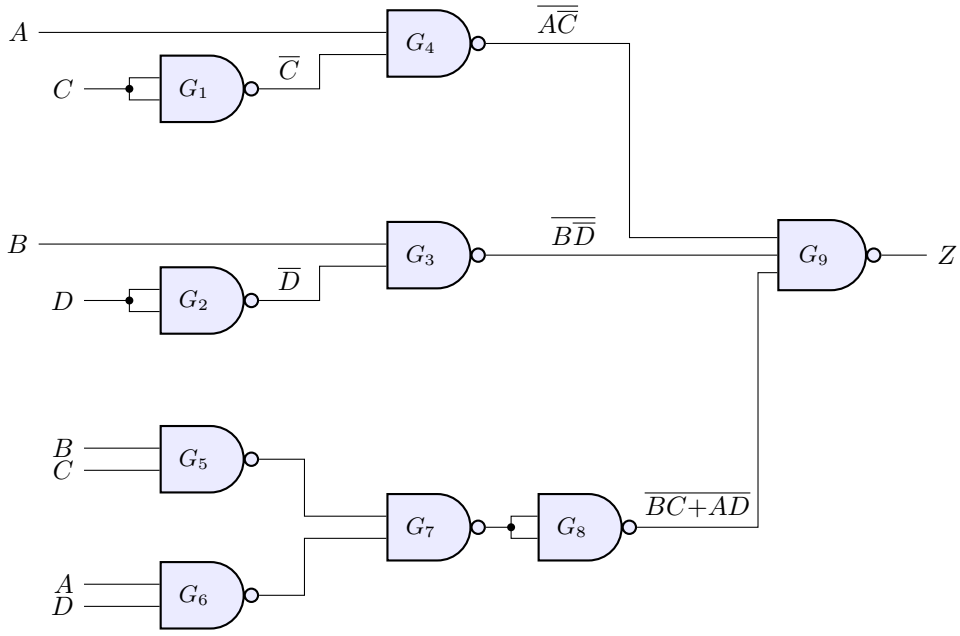
(4 terms, 8 literals, compared with 5 terms in the unsimplified expansion of part (b).)

(e) NAND-Only Implementation

A two-level NAND–NAND form would need a 4-input output NAND, which is not allowed. Instead, the two complement-free terms BC and AD are first OR-ed together, then fed into a 3-input output NAND:

$$Z = B\overline{D} + A\overline{C} + (BC + AD) = \overline{\overline{B\overline{D}} \cdot \overline{A\overline{C}} \cdot \overline{(BC + AD)}}$$

Gate	Function	Output
G_1	NAND(C, C)	$\overline{C}$
G_2	NAND(D, D)	$\overline{D}$
G_3	NAND$(B, \overline{D})$	$\overline{B\overline{D}}$
G_4	NAND$(A, \overline{C})$	$\overline{A\overline{C}}$
G_5	NAND(B, C)	$\overline{BC}$
G_6	NAND(A, D)	$\overline{AD}$
G_7	NAND(G_5, G_6)	$BC + AD$
G_8	NAND(G_7, G_7)	$\overline{BC + AD}$
G_9	3-input NAND(G_3, G_4, G_8)	$B\overline{D} + A\overline{C} + BC + AD = Z$



All 9 gates are NANDs (eight 2-input, one 3-input). Simulating this NAND network in `q2_logic.cpp` confirms it equals the original circuit on all 16 input combinations.

(f) Worst-Case Propagation Delay

Each NAND gate adds 20 ns. Count the gate levels on every input-to-output path:

Path	Gates	Delay
$C \to G_1 \to G_4 \to G_9$	3	60 ns
$D \to G_2 \to G_3 \to G_9$	3	60 ns
$A \to G_4 \to G_9$	2	40 ns
$B \to G_3 \to G_9$	2	40 ns
$B, C \to G_5 \to G_7 \to G_8 \to G_9$	4	**80 ns**
$A, D \to G_6 \to G_7 \to G_8 \to G_9$	4	**80 ns**

The longest (critical) path is, for example,

$$B \;\to\; G_5 \;\to\; G_7 \;\to\; G_8 \;\to\; G_9 \;\to\; Z,$$

which passes through 4 gates, so the worst-case propagation delay is

$$t_{pd} = 4 \times 20 \text{ ns} = \boxed{80 \text{ ns}}.$$